

Lab 7

ส่วนที่ 1: Software Design & Principles Explanation

1.1 สถาปัตยกรรมและ GRASP Patterns

โปรเจกต์นี้แบ่งโครงสร้างออกเป็น 4 ชั้นหลัก ได้แก่ Entity (Game), Repository (GameRepository), Service (GameService) และ Controller (GameController) โดยแต่ละชั้นถูกออกแบบให้สอดคล้องกับหลักการ GRASP (General Responsibility Assignment Software Patterns) ดังนี้

1. Controller Pattern

GameController ทำหน้าที่เป็นตัวรับ HTTP Request จาก Browser เท่านั้น ไม่มีการเขียน business logic หรือคำนวณราคาสวนลอยอยู่ในชั้นนี้เลย ทุก method ในคลาสนี้มีหน้าที่แค่ 3 อย่างคือ รับข้อมูลจาก Request (เช่น @PathVariable, @ModelAttribute), ส่งต่อให้ GameService ประมวลผล และเลือกว่าจะ return ชื่อ Template หรือ redirect ไปหน้าไหน การแยกหน้าที่แบบนี้ทำให้ Controller มีความรับผิดชอบที่ชัดเจนตามหลัก Controller Pattern คือเป็น "ตัวกลางที่ประสานงาน" ไม่ใช่ "ตัวที่ลงมือทำงานเอง"

2. High Cohesion

GameService เป็นจุดรวมของ business logic ทั้งหมดที่เกี่ยวข้องกับเกม ไม่ว่าจะเป็นการดึงข้อมูล บันทึกข้อมูล ลบข้อมูล หรือคำนวณราคาสุทธิผ่าน Strategy Pattern ทุก method ในคลาสนี้เกี่ยวข้องกับ "การจัดการข้อมูลเกม" โดยตรงทั้งหมด ไม่มี method ที่ทำหน้าที่อื่นปะปนอยู่ ทำให้คลาสนี้มีความ Cohesion สูง คือรวมสิ่งที่เกี่ยวข้องกันไว้ในที่เดียว ง่ายต่อการทำความเข้าใจและแก้ไข

3. Low Coupling

แต่ละชั้นเชื่อมต่อกันผ่าน Constructor Injection แทนการสร้าง object กันเองตรงๆ (เช่น GameController ไม่รู้จัก GameRepository เลย รู้จักแค่ GameService) ทำให้แต่ละชั้นพึ่งพากันน้อยที่สุด สามารถเปลี่ยนแปลงการทำงานภายในของชั้นหนึ่งได้โดยไม่กระทบชั้นอื่น ตัวอย่างเช่น หากวันหนึ่งต้องเปลี่ยนจาก PostgreSQL เป็นฐานข้อมูลอื่น ก็แก้ไขแค่ GameRepository โดยที่ GameService และ GameController ไม่ต้องแก้ไขเลย

4. Indirection

DiscountContext ทำหน้าที่เป็นตัวกลาง (Indirection) ระหว่าง GameService กับคลาส Strategy ทั้งสามตัว (NoDiscountStrategy, StudentDiscountStrategy, SeasonalSaleStrategy) โดย GameService ไม่จำเป็นต้องรู้เลยว่า Strategy ที่ตัว หรือ Strategy แต่ละตัวคำนวณอย่างไร เพียงแค่ส่ง discountType และราคาไปให้ DiscountContext แล้วรอรับราคาสุทธิกลับมานั่นเอง การมีตัวกลางแบบนี้ช่วยลดการพึ่งพาโดยตรงระหว่าง Service กับรายละเอียดการคำนวณส่วนลด

5. Information Expert

Game (Entity) เป็นผู้เก็บข้อมูลของตัวเองทั้งหมด (title, genre, platform, rating, price, discountType) และมี getter/setter ที่ให้ชั้นอื่นเข้าถึงข้อมูลได้อย่างเป็นระเบียบ ตามหลัก Information Expert คือให้คลาสที่ "รู้ข้อมูล" มากที่สุดเป็นผู้รับผิดชอบข้อมูลนั้น

1.2 High-Level SOLID Principles

S — Single Responsibility Principle (SRP)

แต่ละคลาสในระบบมีหน้าที่เดียวที่ชัดเจน เช่น GameRepository รับผิดชอบแค่การติดต่อฐานข้อมูล, DiscountContext รับผิดชอบแค่การเลือก Strategy ที่เหมาะสม, GameController รับผิดชอบแค่การจัดการ HTTP Request/Response ไม่มีคลาสใดทำหน้าที่ซ้อนทับกัน

O — Open/Closed Principle (OCP)

ระบบ Strategy Pattern ถูกออกแบบให้ "เปิดสำหรับการขยาย แต่ปิดสำหรับการแก้ไข" หากในอนาคตต้องเพิ่มโปรโมชั่นใหม่ เช่น ส่วนลดวันเกิด 15% เพียงแค่สร้างคลาสใหม่ implement DiscountStrategy แล้วเพิ่ม case ใน DiscountContext เท่านั้น โดยไม่ต้องแก้ไขโค้ดของ NoDiscountStrategy, StudentDiscountStrategy, SeasonalSaleStrategy หรือ GameService ที่มีอยู่เดิมเลย

L — Liskov Substitution Principle (LSP)

ทุกคลาสที่ implement DiscountStrategy (ทั้ง NoDiscountStrategy, StudentDiscountStrategy, SeasonalSaleStrategy) สามารถใช้แทนกันได้อย่างสมบูรณ์ผ่าน interface เดียวกัน โค้ดในส่วนที่เรียกใช้งาน (DiscountContext) ไม่จำเป็นต้องรู้ว่ากำลังใช้ Strategy ตัวไหนอยู่ เพราะทุกตัวมีพฤติกรรมสอดคล้องกับสัญญาที่ interface กำหนดไว้ (รับราคา คืนราคาสุทธิ)

I — Interface Segregation Principle (ISP)

DiscountStrategy มีเพียง method ที่จำเป็นจริงๆ 2 ตัวคือ calculateDiscount() และ getDisplayName() ไม่มีการขัดแย้ง method ที่ไม่เกี่ยวข้องเข้าไปใน interface ทำให้คลาสที่ implement ไม่ต้องเขียนโค้ดที่ไม่จำเป็น

D — Dependency Inversion Principle (DIP)

GameService ไม่ได้พึ่งพาคลาส Strategy ที่เป็นรายละเอียดปลีกย่อยโดยตรง แต่พึ่งพาผ่าน DiscountContext ซึ่งภายในใช้ type เป็น DiscountStrategy (interface) ไม่ใช่คลาสถูกโดยตรง ทำให้ชั้นบน (Service) ไม่ผูกติดกับการ implement ของชั้นล่าง (Strategy classes) สอดคล้องกับหลักการที่ว่า "โมดูลระดับสูงไม่ควรพึ่งพาโมดูลระดับต่ำ ทั้งสองควรพึ่งพา abstraction"

1.3 Strategy Pattern ในการคำนวณส่วนลดราคาเกม

ระบบคำนวณส่วนลดราคาเกมถูกออกแบบด้วย Strategy Pattern ประกอบด้วย 5 ส่วนหลัก ได้แก่

- DiscountStrategy - interface กลางที่กำหนดสัญญาว่า Strategy ทุกตัวต้องมี method calculateDiscount(double price) สำหรับคำนวณราคาสุทธิ และ getDisplayName() สำหรับคืนชื่อโปรโมชั่นที่อ่านง่าย
- NoDiscountStrategy - คืนราคาเดิม ไม่มีส่วนลด ใช้เมื่อ discountType เป็นค่าว่างหรือไม่ตรงกับโปรโมชั่นใด
- StudentDiscountStrategy - คำนวณราคาหลังหักส่วนลด 10% สำหรับนักศึกษา
- SeasonalDiscountStrategy - คำนวณราคาหลังหักส่วนลด 20% สำหรับโปรโมชั่นเทศกาล
- DiscountContext - ตัวกลางที่รับค่า discountType (String) แล้วใช้ switch-case เลือกว่าจะสร้างและเรียกใช้ Strategy ตัวไหน ก่อนคืนผลลัพธ์กลับไปให้ผู้เรียก

เหตุผลที่เลือกใช้ Strategy Pattern แทนการเขียน if-else ขาวๆ ใน Service โดยตรงคือเพื่อให้ระบบยึดหลัก Open/Closed Principle (OCP) หากในอนาคตอาจมีหรือผู้ใช้งานต้องการเพิ่มโปรโมชั่นใหม่ ผู้พัฒนาสามารถสร้างคลาสใหม่ที่ implement DiscountStrategy แล้วเพิ่ม case เดี่ยวใน DiscountContext โดยไม่จำเป็นต้องแก้ไข ทดสอบซ้ำ หรือเสี่ยงทำให้โค้ดเดิมที่ทำงานถูกต้องอยู่แล้วพังลง ซึ่งต่างจากการเขียน if-else ขาวๆ ที่ทุกครั้งที่มีเงื่อนไขใหม่ต้องเข้าไปแก้ไขบล็อกลำดับเดิมโดยตรง เพิ่มความเสี่ยงต่อการเกิด bug ในส่วนที่ไม่เกี่ยวข้อง

1.4 Layered Architecture

ระบบนี้แบ่งออกเป็น Presentation Layer (Controller + Template), Business Logic Layer (Service + Strategy), และ Data Access Layer (Repository) โดยจริงแยก Service Layer ออกมาต่างหาก ไม่ให้ Controller เรียก Repository ตรงๆ ด้วยเหตุผลดังนี้

Low Coupling

หาก Controller เรียก Repository โดยตรง Controller จะผูกติดกับรายละเอียดของการเข้าถึงฐานข้อมูลไปด้วย

เมื่อใดที่ต้องเปลี่ยนวิธีการคำนวณส่วนลด หรือเปลี่ยนแหล่งข้อมูล จะกระทบ Controller โดยตรง แต่เมื่อมี Service คั่นกลาง Controller จะรู้จักแค่ "สิ่งที่ Service ทำได้" (เช่น getAllGames(), saveGame()) โดยไม่ต้องรู้ว่าข้างในมีการคำนวณหรือติดต่อกับฐานข้อมูลอย่างไร

High Cohesion

business logic ทั้งหมด (การคำนวณราคาสุทธิผ่าน Strategy Pattern) ถูกรวมไว้ที่ Service เพียงจุดเดียว ไม่กระจัดกระจายอยู่ใน Controller หรือ Template ทำให้ง่ายต่อการทดสอบและแก้ไข เพราะรู้แน่ชัดว่าต้องไปแก้ที่ไฟล์ไหน

แยกความรับผิดชอบชัดเจน (Separation of Concerns)

Controller จัดการเรื่อง HTTP (routing, request/response) ส่วน Repository จัดการเรื่องฐานข้อมูล (SQL, transaction) โดยมี Service เป็นชั้นที่ประสานทั้งสองฝั่งเข้าด้วยกันพร้อมเพิ่มเติม business logic ที่จำเป็น ทำให้แต่ละชั้นสามารถถูกทดสอบและแก้ไขแยกจากกันได้อย่างอิสระ

1.5 Execution Flow (ลำดับการทำงานเมื่อมี HTTP Request เข้ามา)

ตัวอย่าง flow การทำงานเมื่อผู้ใช้เปิดหน้ารายการเกมทั้งหมด (GET /games)

1. Browser ส่ง HTTP Request แบบ GET มาที่ /games
2. DispatcherServlet ของ Spring รับ Request แล้วส่งต่อไปยัง method ที่ตรงกับ route นี้ใน GameController (คือ method ที่มี @GetMapping ตรงกับ /games)
3. GameController เรียก gameService.getAllGames() โดยไม่ทำ logic ใดๆ เอง
4. GameService เรียก gameRepository.findAll() เพื่อดึงข้อมูลเกมทั้งหมดจากฐานข้อมูล PostgreSQL ผ่าน Spring Data JPA (Hibernate จะแปลงเป็นคำสั่ง SQL SELECT โดยอัตโนมัติ)
5. เมื่อได้ข้อมูลดิบกลับมา GameService จะวนลูปในแต่ละเกม แล้วเรียก discountContext.applyDiscount(price, discountType) และ discountContext.getDisplayName(discountType) เพื่อคำนวณราคาสุทธิและชื่อโปรโมชัน
6. DiscountContext จะเลือก Strategy ที่ตรงกับ discountType ของเกมนั้น (เช่น SeasonalSaleStrategy หากเป็นโปรโมชันเทศกาล) แล้วเรียก method คำนวณของ Strategy ตัวนั้น คืนผลลัพธ์กลับไปยัง Service
7. GameService เซ็ตค่าราคาสุทธิและชื่อโปรโมชันกลับเข้าไปใน object Game แต่ละตัว แล้ว return list ทั้งหมดกลับไปยัง Controller
8. GameController นำข้อมูลที่ได้อัดลงใน Model ด้วยชื่อ attribute "games" แล้ว return ชื่อ Template "games/list"
9. Thymeleaf Template Engine นำ list.html มาประมวลผลร่วมกับข้อมูลใน Model โดยวนลูปแสดงแต่ละเกมในตาราง คำนวณและ render เป็นไฟล์ HTML
10. Spring ส่ง HTML ที่ render เสร็จแล้วกลับไปยัง Browser เพื่อแสดงผลให้ผู้ใช้เห็น

สำหรับกรณีที่มีการบันทึกข้อมูล (POST /games/save) flow จะคล้ายกันแต่สลับทิศทาง คือรับข้อมูลจากฟอร์มผ่าน @ModelAttribute Game game เข้ามาที่ Controller ส่งต่อให้ Service เรียก gameRepository.save(game) เพื่อบันทึกลงฐานข้อมูล (Hibernate แปลงเป็นคำสั่ง INSERT หรือ UPDATE โดยอัตโนมัติขึ้นอยู่กับว่ามี id อยู่แล้วหรือไม่) จากนั้น Controller จะ redirect กลับไปที่ /games เพื่อให้ผู้ใช้เห็นรายการเกมล่าสุด

ส่วนที่ 2: Code Implementation & Dependency Injection

ทุกชั้นในระบบใช้ Constructor Injection ในการเชื่อมต่อกัน แทนการใช้ @Autowired แบบใส่ field ตรงๆ หรือการ new object เองโดยตรง ด้วยเหตุผลดังนี้

GameController มี constructor รับ GameService เข้ามา

```
public GameController(GameService gameService) {
    this.gameService = gameService;
}
```

Spring จะมองเห็นว่า GameController ต้องการ GameService แล้วจะไปหา Bean ของ GameService ที่สร้างไว้แล้ว (จาก @Service) มาใส่ให้อัตโนมัติตอนสร้าง object โดยที่ผู้พัฒนาไม่ต้องเขียนคำสั่ง new GameService() เองเลย

GameService มี constructor รับทั้ง GameRepository และ DiscountContext

```
public GameService(GameRepository gameRepository, DiscountContext discountContext) {
    this.gameRepository = gameRepository;
    this.discountContext = discountContext;
}
```

ทั้งสอง dependency ถูก inject เข้ามาพร้อมกันตอนสร้าง Bean ทำให้ GameService ไม่ต้องรู้วิธีสร้าง object เหล่านี้เอง เพียงแค่ "ขอใช้งาน" ผ่าน constructor เท่านั้น

ประโยชน์ของ Constructor Injection ในภาพรวม

- Field เป็น final ได้ — ทำให้ dependency ไม่สามารถถูกเปลี่ยนแปลงได้หลังจากสร้าง object แล้ว ลดโอกาสเกิด bug จากการเปลี่ยนค่ากลางทาง
- ทดสอบง่ายขึ้น — เวลาเขียน Unit Test สามารถส่ง Mock object เข้าไปทาง constructor ได้โดยตรง โดยไม่ต้องพึ่งพา Spring Context จริง
- มองเห็น dependency ได้ชัดเจน — แคล้ว constructor ก็รู้ทันทีว่าคลาสนี้ต้องพึ่งพาอะไรบ้าง ต่างจาก @Autowired แบบ field ที่ต้องไล่อ่านทั้งคลาสถึงจะรู้
- สอดคล้องกับ Dependency Inversion Principle (DIP) — เพราะ dependency ถูกกำหนดจากภายนอก (Spring เป็นคนใส่ให้) ไม่ใช่คลาสสร้างขึ้นเอง ทำให้แต่ละชั้นไม่ผูกติดกับ implementation รายละเอียดของอีกชั้นหนึ่งโดยตรง